

Overview

For Sprint 3, our team identified the stories and spikes with the strongest dependencies and organized them into a network diagram. The purpose of the diagram is to show the order of dependent product work, identify the critical path, and understand which delays would have the biggest effect on the sprint schedule.

The sprint goal was to complete seller offer decisions, transaction creation and history, buyer and seller ratings, user profiles, and messaging. We also included spike tasks for areas that required research before implementation.

Assumptions Used

For the network diagram, we used the following assumptions:

- user stories and spikes were labelled with story points as rough effort estimates
- only product stories and supporting spike work were included in the chart
- Docs + Design were intentionally removed from the schedule diagram
- dependencies were based on likely implementation order
- the diagram was created at the story and spike level rather than for every small checklist task
- some branches are close in total duration, so the sprint may have more than one critical branch

Tasks with Dependencies

The main sprint tasks were:

- US14: Reject Offer
- US15: Accept Offer
- US17: Rate Seller
- US18: Rate Buyer
- US19: View Users' Profiles with Ratings
- US20: Message a Seller
- US21: View Messages
- US22: View Transaction History

The supporting spike tasks were:

- Spike 1: Transaction linkage research
- Spike 2: Ratings research
- Spike 3: Messaging research

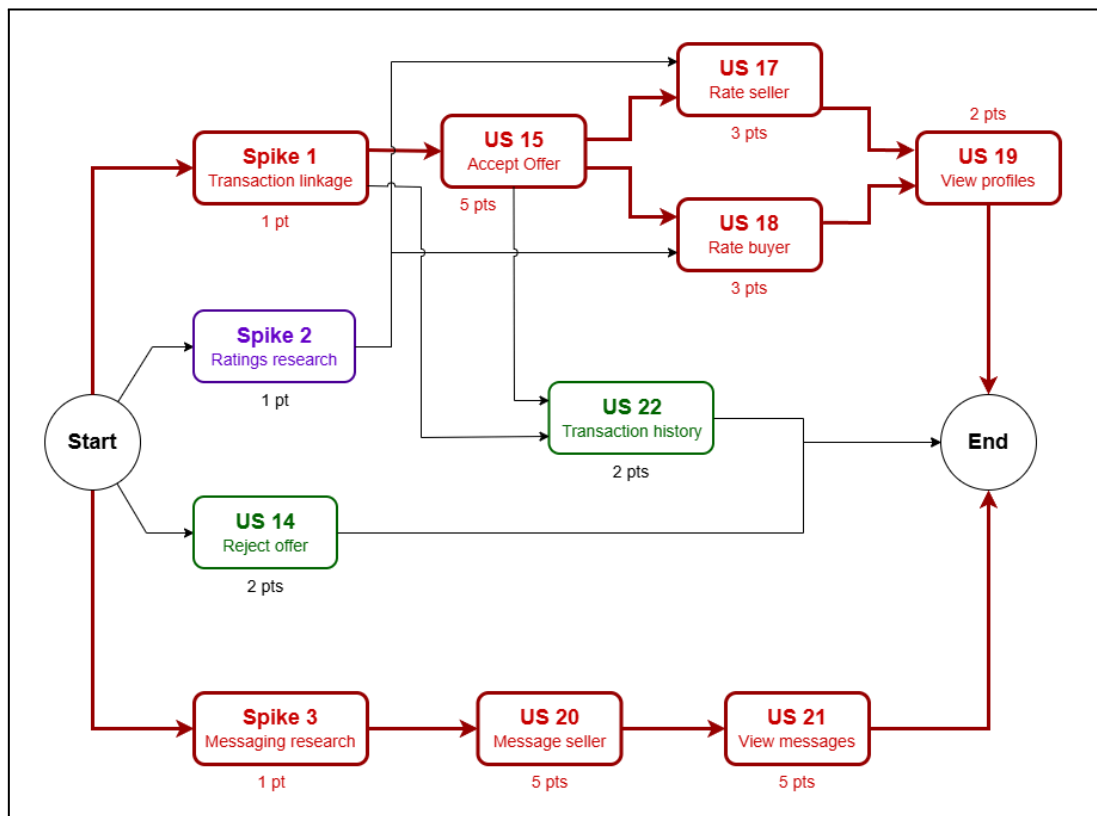
The main dependencies were:

- US15 depends on Spike 1 because accepting an offer also creates the transaction record and updates listing and offer state

- US17 and US18 depend on US15, and are supported by Spike 2, because ratings happen only after an accepted transaction
- US19 depends on US17 and US18 because profile reputation data comes from buyer and seller ratings
- US20 depends on Spike 3 for the message schema, storage, and conversation design
- US21 depends on US20 because viewing conversations requires existing messages and reply flow
- US22 depends on Spike 1 and US15 because transaction history only becomes meaningful after accepted offers create transaction records

Network Diagram

The critical paths are shown in red.



Critical Path

In this diagram, there are two main critical branches:

- Start → Spike 1 → US15 → US17/US18 → US19 → End
- Start → Spike 3 → US20 → US21 → End

These branches are critical because they contain the longest sequences of dependent product work (11pts) and therefore have the greatest impact on the overall sprint schedule. The ratings and profile branch is especially important because US19 depends on both buyer-side and seller-side reputation data, while the messaging branch is a long independent sequence with little slack.

How We Plan to Stay on Schedule

To keep the sprint on schedule, we planned to complete the spike tasks early so that the more dependent stories could begin sooner. Transaction linkage research reduces uncertainty before acceptance and history work, ratings research supports both rating stories, and messaging research helps unblock the message and inbox features.

We also divided work across separate branches where possible. Offer handling and transaction history, ratings and profiles, and messaging can proceed mostly in parallel once the relevant spike work is done. This allows team members to work in parallel rather than waiting on a single sequence of tasks.

We paid special attention during standups to the acceptance-to-ratings-to-profile branch and the messaging branch because they were on the critical path. These parts of the sprint had the highest chance of creating delays due to data-model and integration work.

Reflection

Compared with the previous sprint schedule, this version focuses only on product stories and supporting spike work that directly affect delivery. Removing Docs + Design from the network diagram makes the critical path easier to justify and aligns the schedule more closely with the feedback from the last sprint.

The diagram also shows that Sprint 3 had two main risk areas: reputation features and messaging. Mapping those dependencies in advance helped clarify where delays would matter most and where work could still proceed in parallel.